SW개발 보안 구축

KOREA
코리아IT아카데미학원



class	정보보안 산업기사 취득과정 A
name	조채언

목 차

Ⅰ. SW 개발보안의 3요소 ····· 3

Ⅱ. SW개발보안의 개념도 ····· 3

Ⅲ. 소프트웨어 개발보안 계획 수립 ····· 4

가. 소프트웨어 개발보안 계획 ····· 4

나. 소프트웨어 보안 요구공학 프로세스 개념도 ····· 4

Ⅳ. 보안성 강화된 응용프로그램 구현 계획 수립 ····· 5

Ⅴ. 보안성 강화된 응용프로그램 구현 환경 구축 ····· 5

가. 보안취약점 환경구축 ····· 5

나. 취약점 모니터링 환경 ····· 5

Ⅵ. SW개발보안 구현을 위한 시큐어 코딩 ····· 6

Ⅶ. 응용프로그램 구현 환경 관리 ····· 13

Ⅷ. 구현된 응용프로그램 결함 여부 테스트 ····· 14

가. 일반적인 보안 테스트 절차 ····· 14

나. 보안 정적/동적 분석 ····· 14

Ⅸ. 보안결함 재발방지 계획 ····· 16

Ⅹ. 정보보호 시스템 로그 점검 관리대장 작성 ····· 19

Ⅺ. 로그관리 점검 결과 ····· 20

Ⅰ. SW 개발보안의 3요소

항목	내용
기밀성 (confidentiality)	• 정보가 인가된 사용자나 시스템만이 접근할 수 있어야 함을 의미 • 이는 정보가 무단으로 공개되거나 유출되는 것을 방지하는 것 • 민감한 정보의 노출을 방지하고, 인가되지 않은 엑세스로부터 데이터를 보호
무결성 (integrity)	• 정보는 인가된 사용자와 정당한 절차에 의해서만 수정·변경되어야 함을 의미 • 데이터가 손상되거나 조작되지 않아야 함 • 정보의 정확성과 신뢰성을 보장하고, 데이터의 무결성을 유지
가용성 (availability)	• 정보나 시스템이 필요한 시간에 접근 가능하고 사용 가능해야 함을 의미 • 시스템의 정상적인 작동을 보장하고, 서비스 거부 공격 등으로부터 시스템을 보호하여 사용자가 원하는 시간에 서비스를 이용할 수 있도록 하는 것

Ⅱ. SW개발보안의 개념도

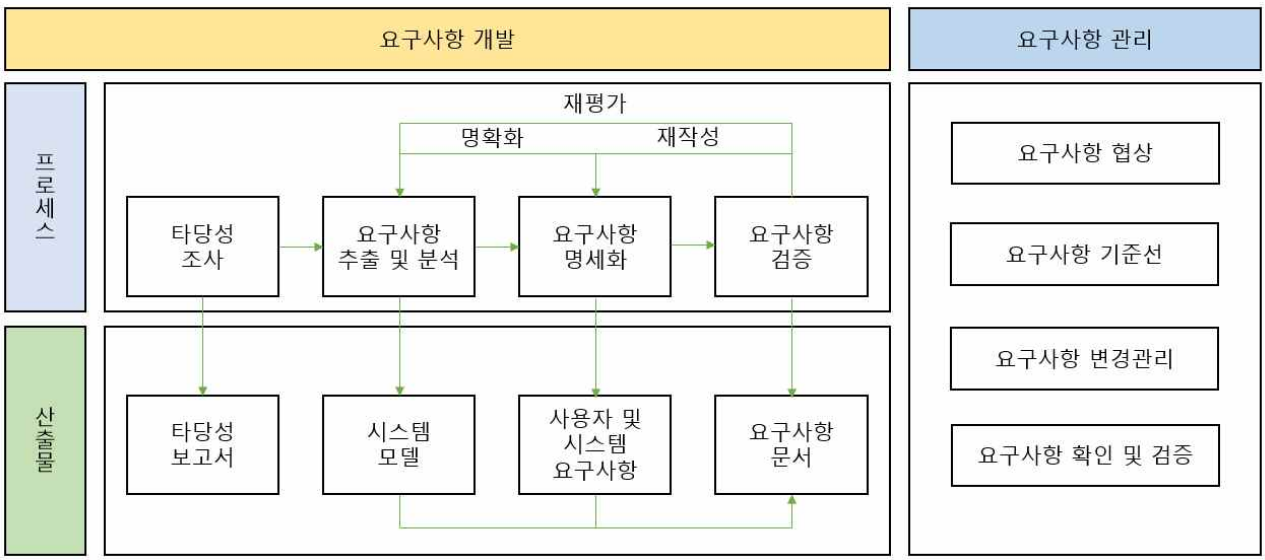
항목	설명
자산 (Asset)	조직 또는 기관에서 보유하고 있는 정보화 자산 및 데이터 (서버, 네트워크, 소프트웨어, 중요 데이터 등)
위협원 (Threat agents)	자산에 피해를 줄 수 있는 내·외부 공격 위험 주체 (인가받지 않은 직원, 외부 공격 해커, 자연재해 등)
위협 (Threat)	자산에 피해를 주는 위협원의 공격 행동 (해킹 등 사이버공격, 자신의 위변조 및 삭제, 중요 자산의 불법 유통 등)
취약점 (Vulnerability)	위협이 발생할 수 있는 상황 (Injection, 웹 취약점, 취약한 암호화, 사용하지 않은 포트 오픈 등)
위험 (Risk)	위협원이 취약점을 이용하여 자산에 위해를 입힐 수 있는 상황 예측

Ⅲ. 소프트웨어 개발보안 계획 수립

가. 소프트웨어 개발보안 계획

단계	활동	산출물
요구사항 분석	• 요구사항 중 보안 요구 항목 식별 • 시스템 보안 항목 식별 • 개발 환경의 보안 프레임워크 분석 • 운영 보안 환경 분석	• 보안요구사항 정의서 • 보안적용계획서
설계	• 보안 요구 항목 구현 방안 설계 • 보안 프레임워크 인증, 인가 설정 • 화면 설계 등 보안 적용 계획 반영 설계	• 보안적용계획서 • 보안 프레임워크
구현	• 개발 표준 및 SW개발 보안 가이드를 준수한 소스코드 작성(secure coding) • 소스코드 보안약점 진단 및 조치 • 코드 리뷰(code review)	• 보안약점 진단 결과서 및 조치 결과 • 보안약점이 제거된 소스코드
테스트	• 오류를 유발할 수 있는 내용이 포함된 테스트 케이스 기반 테스트 • 공격 시나리오를 포함한 테스트 수행 • 취약점 등 모의 침투 테스트 수행	• 테스트 결과서 • 결함 보고서 • 취약점 조치 결과서
운영 및 유지보수	• 지속적인 보안 패치 • 보안약점, 취약점 진단 및 조치	• 보안 패치 결과서 • 보안약점, 취약점 진단 결과 및 조치 결과서

나. 소프트웨어 보안 요구공학 프로세스 개념도



Ⅳ. 보안성 강화된 응용프로그램 구현 계획 수립

1. **일정 계획을 위한 보안 전문가 선정**
   개발 부서에서는 개발을 위한 모든 과정(계획, 요구사항 분석, 도출, 프로그램 소스코드 선택, 계약과 발주, 프로젝트 관리 등)은 지식이 풍부하고, 동일 업무에 경험이 있는 공신력 있는 보안 전문가와 보안 실무자를 통해 수행될 수 있도록 해야 한다.

2. **일정 계획을 위한 보안 범주 식별**
   조직의 보안 범위는 개발되는 시스템의 데이터와 정보처리시스템을 위협하는 사고의 잠재적 파급 효과를 기반으로 조직의 보안요구사항을 도출하여 선택하고 내용은 일정에 반영한다.

3. **보안요구사항 분석**
   중요 보안 고려 사항뿐 아니라 소프트웨어 개발자와 공급자 선택, 조직의 각 주요 부서 간의 역할 및 책임, 전사적인 프로젝트 관리 등에 대한 사항이 기술되어야 한다.

4. **보안성이 강화되었는지 취약점 점검과 함께 독립적인 테스트 계획 기획**
   보안이 강화된 소프트웨어 구축, 안정성 평가 기능과 특별한 요구사항이 있는지 직간접적으로 확인하기 위한 취약점 점검 계획이 기술되어야 한다.

Ⅴ. 보안성 강화된 응용프로그램 구현 환경 구축

가. 보안취약점 환경구축

1. **대상 시스템을 테스트 환경에 배포**
   운영 서비스와 분리된 별도의 독립 테스트 환경을 구축한다.
2. **보안 취약점 점검에 필요한 진단 도구와 모니터링 툴을 설치 관리**
   취약점 분석 및 모니터링을 위한 필수 솔루션을 구성하고 유지한다.
3. **보안 취약점 진단하는 환경 설정 정보를 문서화 명세 관리**
   환경 설정과 기술 명세를 기록하여 점검의 일관성을 확보한다.

※ 취약점 진단을 위한 하드웨어 환경 구성

하드웨어	설명
진단 서버	취약점 진단을 수행하기 위해 필요한 CPU, 메모리 등 각종 하드웨어가 장착된 서버이다.
스토리지	취약점 진단을 수행하기 위해 필요한 데이터를 담아 두는 저장 공간으로 이러한 데이터를 취약점 진단 데이터라고 한다.
네트워크	서버와 클라이언트를 연결해 주는 연결망으로 기업 내부에서는 근거리 유선망 (LAN, Local Area Network)로 구성된다.

나. 취약점 모니터링 환경

1. **보안 취약점 진단 진행 중 보안 취약점 진단 환경 인프라와 대상 시스템에 대한 모니터링 수행**
   진단 중 발생하는 시스템 부하와 인프라 이상 유무를 실시간 감시합니다.
2. **보안 취약점 진단 진행 중 환경에 대한 모니터링 정보를 저장**
   진단 과정의 로그와 데이터를 수집하여 분석 및 증거 자료로 활용합니다.

VI. SW개발보안 구현을 위한 시큐어 코딩

1. flawfinder 설치

Flawfinder는 C 소스 코드에서 보안 취약점을 탐지하는 정적 분석 도구로, 위험 함수 사용 여부를 분석하여 버퍼 오버플로우 등 보안 결함을 사전에 발견하기 위해 설치한다.

```
root@kali:/home/kali# apt install flawfinder

다음 패키지가 자동으로 설치되었지만 더 이상 필요하지 않습니다:
  curlftpfs                 libfuse2t64               librubberband2
  fcitx-frontend-all        libgail-common            libsframe2
  fcitx-frontend-gtk2       libgail18t64              libsimdutf27
  fcitx-frontend-gtk3       libgav1-1                 libsnmp40t64
  fcitx-frontend-qt6        libgdk-pixbuf2.0-bin      libsphinxbase3t64
  fcitx-ui-classic          libgettextpo0             libswscale8
  gir1.2-fcitx-1.0          libgtk2.0-0t64            libtsk19t64
  ibus-gtk                  libgtk2.0-bin             libvdpau-va-gl1
  libann0                   libgtk2.0-common          pocketsphinx-en-us
  libaudio2                 liblab-gamut1             python3-aiocache
  libavfilter10             libmjpegutils-2.1-0t64    python3-aiomcache
  libavformat61             libmpeg2encpp-2.1-0t64    python3-fs
  libcdt5                   libmplex2-2.1-0t64        python3-terminaltables
  libcgraph6                libmupdf25.1              python3-wapiti-arsenic
  libconfig-inifiles-perl   libplacebo351             python3-yaswfp
  libcrypt-dev              libpocketsphinx3          ruby-unf-ext
```

2. test.c 파일 작성

vim을 실행하여 시큐어 코딩을 작성할 c파일을 연다.

```
root@kali:/home/kali# vim test.c
```

3. 안전하지 않은 코드 구현

코드에서는 scanf("%s", buf)를 사용하여 사용자 입력을 처리하고 있다. 이 함수는 입력 길이에 대한 제한이 없기 때문에, 버퍼의 크기를 초과하는 입력이 들어올 경우 문제가 발생할 수 있다.

※ 취약한 이유

가. 입력 길이 제한이 없다.

- scanf("%s")는 입력 데이터의 길이를 제한하지 않는다.
- buf는 10바이트로 제한되어 있다.

나. 버퍼 오버플로우 발생할 수 있다.

- 사용자가 10자 이상의 문자열 입력 시 배열 범위를 초과하여 메모리를 침범할 수 있다.

다. 메모리 손상 가능성이 있다.

- 인접한 변수 또는 프로그램 흐름이 변조될 수 있다.
- 심한 경우 프로그램 비정상 종료된다. (Segmentation Fault)

라. 악의적 공격 가능

- 공격자가 의도적으로 긴 입력을 넣을 경우 프로그램 제어 흐름이 변경될 가능성이 존재한다.

```c
#include <stdio.h>

int main() {
    char buf[10];
    scanf("%s", buf);
    printf("%s\n", buf);
    return 0;
}
```

4. 안전하지 않은 코드 정적 분석

코드를 정적 분석 도구인 Flawfinder를 사용하여 보안 취약점을 점검하였다.

※ 발견된 취약점

- scanf("%s", buf)
  입력 길이에 대한 제한이 없어 버퍼 오버플로우 취약점 발생 가능성이 있다.
- char buf[10]
  고정 크기 배열 사용으로 인해 입력 데이터가 배열 크기를 초과할 경우 메모리가 손상될 가능성이 있다.

```
root@kali:/home/kali# flawfinder test.c
Flawfinder version 2.0.19, (C) 2001-2019 David A. Wheeler.
Number of rules (primarily dangerous function names) in C/C++ ruleset: 222
Examining test.c

FINAL RESULTS:

test.c:5:  [4] (buffer) scanf:
  The scanf() family's %s operation, without a limit specification, permits
  buffer overflows (CWE-120, CWE-20). Specify a limit to %s, or use a
  different input function.
test.c:4:  [2] (buffer) char:
  Statically-sized arrays can be improperly restricted, leading to potential
  overflows or other issues (CWE-119!/CWE-120). Perform bounds checking, use
  functions that limit length, or ensure that the size is larger than the
  maximum possible length.

ANALYSIS SUMMARY:

Hits = 2
Lines analyzed = 8 in approximately 0.00 seconds (2980 lines/second)
Physical Source Lines of Code (SLOC) = 7
Hits@level = [0]   1 [1]   0 [2]   1 [3]   0 [4]   1 [5]   0
Hits@level+ = [0+]   3 [1+]   2 [2+]   2 [3+]   1 [4+]   1 [5+]   0
Hits/KSLOC@level+ = [0+] 428.571 [1+] 285.714 [2+] 285.714 [3+] 142.857 [4+] 142.857 [5+]   0
Minimum risk level = 1

Not every hit is necessarily a security vulnerability.
You can inhibit a report by adding a comment in this form:
// flawfinder: ignore
Make *sure* it's a false positive!
You can use the option --neverignore to show these.

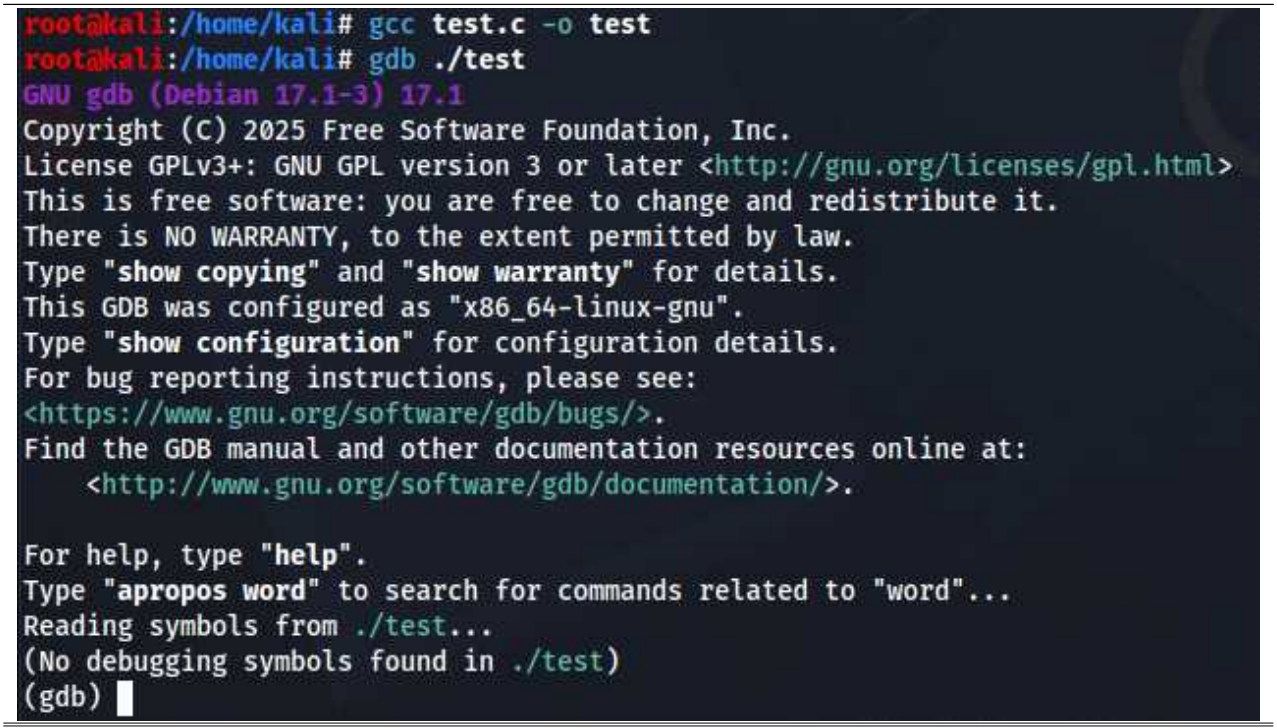
There may be other security vulnerabilities; review your code!
See 'Secure Programming HOWTO'
(https://dwheeler.com/secure-programs) for more information.
```

5. 안전하지 않은 코드 실행 및 동적 분석

안전하지 않은 코드의 실행 및 동적 분석을 위해 GCC 컴파일러를 사용하여 실행 파일을 생성하고, GNU Debugger(GDB)를 실행하였다.

※ GDB 실행 결과

- 디버깅 심볼이 포함되지 않았다는 메시지 (No debugging symbols found) 가 출력되었다.
- 이는 단순 실행 파일이며, 디버깅 옵션(-g) 없이 컴파일되었음을 의미한다.



6. 시큐어 코딩 (안전한 코드) 적용

취약한 기존 코드를 개선하기 위해 다음과 같은 시큐어 코딩을 적용하였다.

※ 안전한 이유

1. 입력 길이 제한 (fgets(buf, 100, stdin))
   - 최대 100바이트까지만 입력 허용하여 버퍼 오버플로우를 방지한다.
2. 충분한 메모리 확보 (malloc(100))
   - 입력 크기에 맞는 충분한 공간 확보하여 고정 배열보다 유연하고 안전하다.
3. 메모리 해제 (free(buf))
   - 메모리 누수를 방지해서 안정적인 프로그램을 유지한다.
4. 위험 함수 제거 (gets, scanf("%s") 제거)
   - 대표적인 취약 함수 제거로 보안을 강화하였다.



```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    char *buf = (char *)malloc(100);
    fgets(buf, 100, stdin);
    printf("%s\n", buf);
    free(buf);
    return 0;
}
```

7. 안전한 코드 정적 분석

개선된 코드에 대해 다시 Flawfinder를 실행하여 보안 취약점 여부를 확인하였다.

※ 분석 결과

취약점 탐지 결과 'No hits found.'로 취약점이 없으며, 기존에 발견된 버퍼 오버플로우 문제가 해결되었음을 확인하였다.

```
root@kali:/home/kali# flawfinder test.c
Flawfinder version 2.0.19, (C) 2001-2019 David A. Wheeler.
Number of rules (primarily dangerous function names) in C/C++ ruleset: 222
Examining test.c

FINAL RESULTS:

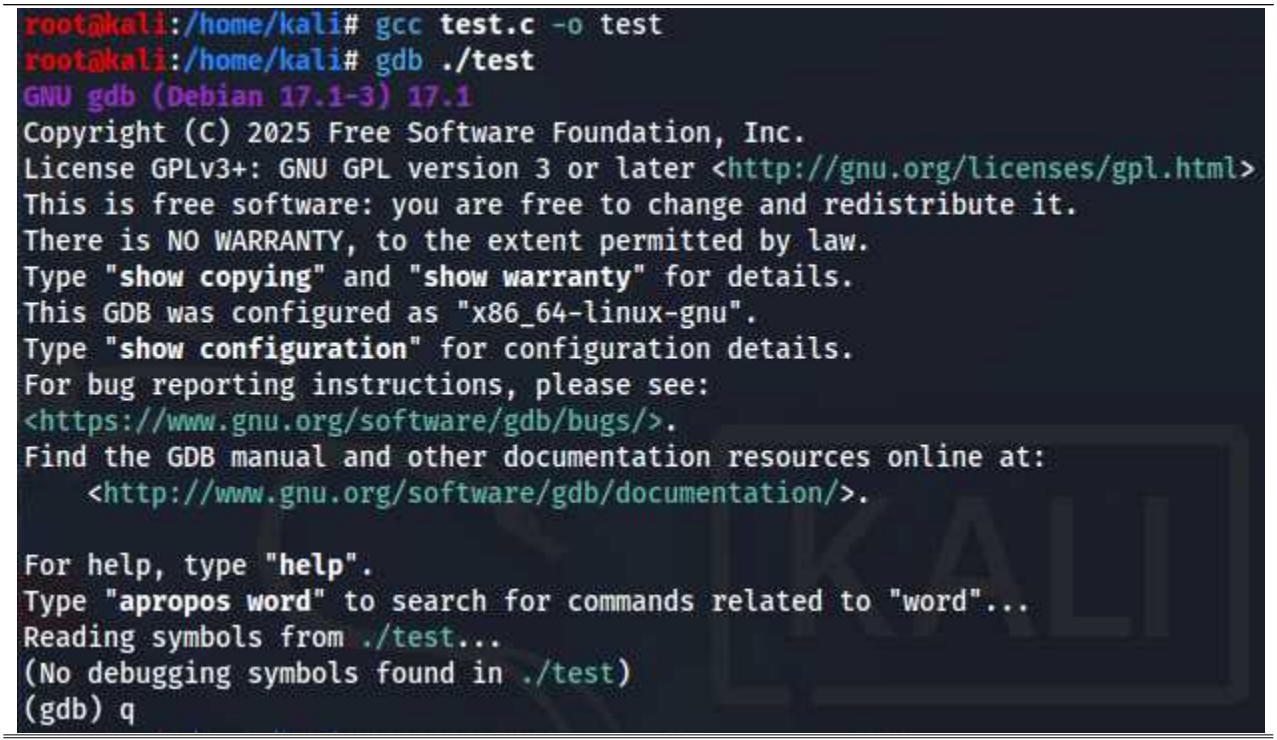

ANALYSIS SUMMARY:

No hits found.
Lines analyzed = 10 in approximately 0.00 seconds (3643 lines/second)
Physical Source Lines of Code (SLOC) = 9
Hits@level = [0]   1 [1]   0 [2]   0 [3]   0 [4]   0 [5]   0
Hits@level+ = [0+]   1 [1+]   0 [2+]   0 [3+]   0 [4+]   0 [5+]   0
Hits/KSLOC@level+ = [0+] 111.111 [1+]   0 [2+]   0 [3+]   0 [4+]   0 [5+]   0
Minimum risk level = 1

There may be other security vulnerabilities; review your code!
See 'Secure Programming HOWTO'
(https://dwheeler.com/secure-programs) for more information.
```

8. 안전한 코드 실행 및 동적 분석

개선된 코드를 컴파일 후 GDB로 실행하여 정상 동작 여부를 확인하였다. 프로그램이 정상적으로 실행되었고, 치명적인 오류 없이 종료가 가능하다.



Ⅶ. 응용프로그램 구현 환경 관리

□ 패스워드 관리

1. **패스워드 최소 길이 제한**
   패스워드의 최소 길이를 통제할 수 있도록 최소 패스워드 길이값을 정의한다.

2. **추측 가능한 패스워드 사용 통제**
   - 사용자 개인정보(주민등록번호, 휴대폰 번호 등)와 관련된 패스워드는 사용하지 못한다.
   - 사용자 ID와 동일한 패스워드는 사용하지 못한다.
   - 동일한 문자 또는 숫자가 반복되는 패스워드는 사용하지 못한다.
   - 문자 또는 숫자만으로 구성된 패스워드는 사용하지 못한다.

3. **패스워드 변경 및 확인 시 보안 적용**
   패스워드 변경 시 현재 패스워드를 확인한 후 변경 사항을 적용한다.

4. **패스워드 파일 접근 통제 및 암호화 저장 여부 진단**
   응용프로그램에 저장된 패스워드 파일은 관리자도 알아볼 수 없도록 암호화된 형태로 저장한다.

5. **이전 부여 패스워드 일정 기간 재사용 통제**
   응용프로그램 인증시스템은 관리자가 이전에 사용했던 패스워드를 일정 기간 동안 재사용하지 못하도록 하는 기능을 구현한다.

6. **패스워드 사용 기간 설정**
   패스워드를 사용할 수 있는 최소 기간을 설정하여 일정 기간이 지나면 개발자로 하여금 강제로 패스워드를 변경하도록 구현한다.

7. **인증(O.T.P, 생체인증, PKI)의 강화 작용**
   전자상거래, 금융 관련 서비스 등 특별하게 인증 강화가 필요할 경우 O.T.P, 생체인증, PKI 등의 강화된 인증 기능을 적용한다.

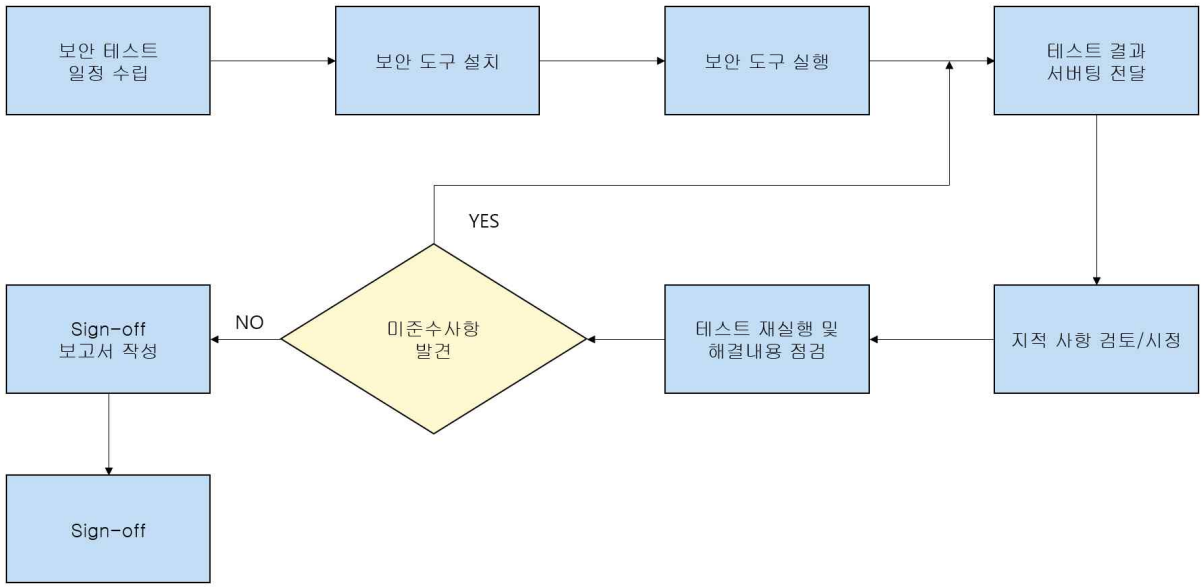
8. **안전한 암호화 알고리즘 사용**
   취약한 암호화 알고리즘이나 충분하지 않은 키 길이 사용으로 인해 무차별 대입 공격(brute-force) 등으로 암호가 노출될 수 있는 보안 위험이 있으므로, 안전한 암호화 알고리즘을 사용한다.

※ 안전한 암호화 알고리즘 권고 사항

대칭키 암호화 알고리즘	비대칭키 암호화 알고리즘
SEED ARIA-128/192/256 AES-128/192/256 Camelia-128/192/256 Blowfish MISTY1 KASUMI 등	RSA KCDSA(전자서명용) RSAES-OAEP ElGamal ECC ECKCSDA 등

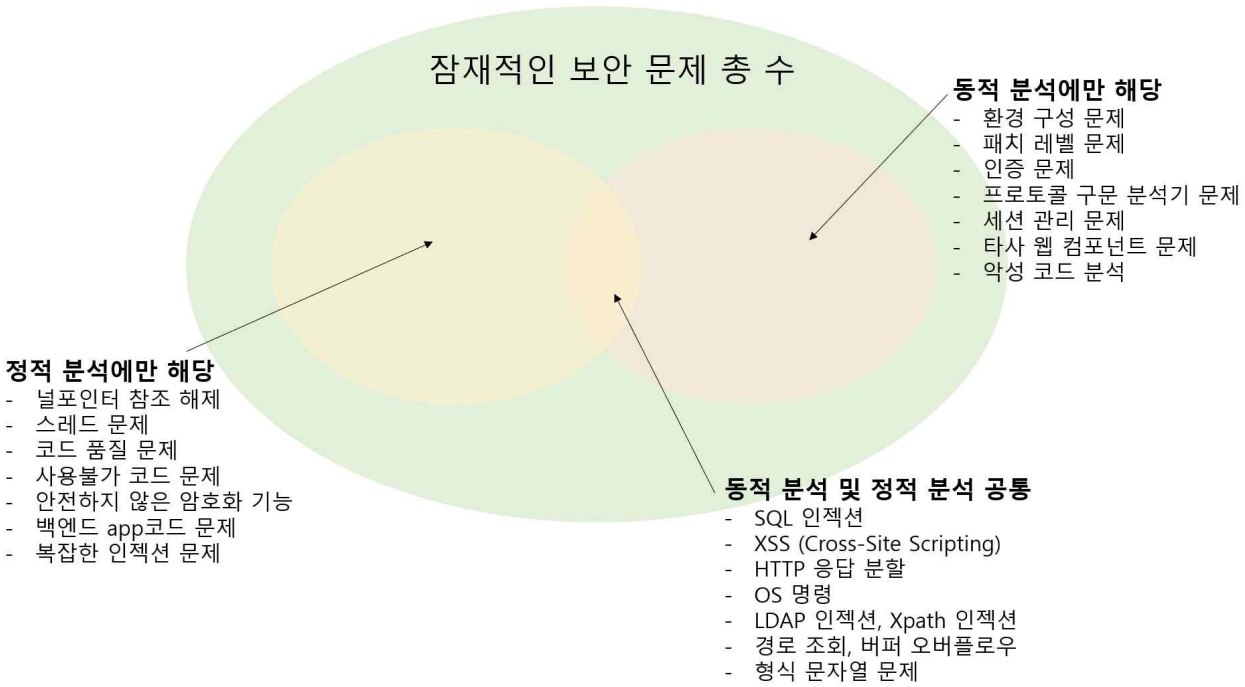
Ⅷ. 구현된 응용프로그램 결함 여부 테스트

가. 일반적인 보안 테스트 절차



나. 보안 정적/동적 분석

※ 정적 분석(SAST)과 동적 분석(DAST)의 취약점 탐지 범위



구분	정적 분석	동적 분석
개념	• 소스코드의 의미를 분석하여 소프트웨어의 결함을 찾아내는 분석기법 • 소프트웨어의 코드 보안약점을 점검하여 완성된 소프트웨어의 발생 가능한 취약점을 예방하는 점검 방법	소프트웨어가 실행중인 환경에서 소프트웨어 소스코드 보다는 실행 과정에서의 다양한 입/출력 데이터의 변화 및 사용자 상호 작용에 따른 변화를 점검하는 분석 기법
주요 기법	• 소스코드 검증 : 검증 가이드라인을 통해 보안 조치 • 코드 리뷰 : 개발자가 작성하고 다른 개발자가 정해진 방법을 통해 검토하는 방법 (동료검토, 제3자검토 라고도 함) • 리버스 엔지니어링 : 시스템의 기술적인 원리를 구조분석을 통해 발견하는 방법	• 디버깅 : 논리적인 오류(버그)를 찾아내는 테스트 과정 • 스트레스 테스트 : 결과 관찰을 목적으로 한계점에 이르는 테스트를 수반 • 모의 해킹 : 내부 또는 외부에서 실제 해커가 사용하는 해킹 도구와 기법 등을 이용하여 정보시스템으로의 침투 가능성을 진단하는 선의의 해킹 기법 • 리버스 엔지니어링 : 동적 역공학 분석툴을 이용하여 구조분석
특징	• 소프트웨어를 실행할 필요가 없음 • 실행 과정/결과 보다는 실행 전 구현에 초점 • 개발초기에 보안 약점의 견으로 인한 수정 비용 절감 효과 • 컴포넌트 간 발생할 수 있는 통합된 보안 약점 발견은 제한적 • 설계, 구조 관점의 보안약점은 발견할 수 없음 • 동적 분석기법과 상호 보완적 기능 수행	• SW를 실행하며 분석 • 실행과정 및 결과에 초점 • 정적 분석에서 발견되지 않는 사용상의 문제점 발견으로 인한 수정비용절감 • 구현 컴포넌트 간 혹은 설계 및 구조관점에서 발생 가능한 보안 약점의 검증 가능 • 정적분석 기법과 상호 보완적 기능 수행
도구	• 컴파일 타임 및 소스 레벨에서 검증 가능한 보안약점 항목 점검 수행 • Coverity Prevent : 코딩과 시스템 빌드 과정 중 소스코드에 게재된 치명적인 버그와 보안 취약점을 자동으로 탐지 • SSEn : 국내 기술의 정적 점검 도구 • Fortify : 다양한 언어를 지원하는 보안 점검 도구	• IBM Rational App Scan • Acunetix 웹 취약 스캐너

IX. 보안결함 재발방지 계획

1. 보안 결함의 심각도를 파악하여 보고 여부를 파악한다.

보안 결함의 심각도가 심각 또는 경계인 경우 사고 대응 책임자가 보안 사고 대응 결과보고서를 작성하고 심각도가 주의 또는 관심인 경우 보안 사고 대응 담당자는 보안 사고 사후 처리 결과서를 작성하여 관리를 수행한다.

가. SW보안 결함의 유형 및 심각도 파악

소프트웨어의 알려진 보안 취약점과 결함의 유형 및 심각도를 파악한다.

나. SW보안 취약점의 제거 확인과 복구 여부 확인

보안 결함에 의해 예측되는 공격에 사용된 보안 취약성이 완전히 제거되었고, 정상서비스 운영 데이터나 파일이 복구되었는지 여부를 파악하고 복구 여부를 확인한다.

다. SW보안 결함의 예상되는 보안 사고의 예측 명기

예상되는 결함과 관련된 모든 업무 담당자에게 보안 결함 분석 결과가 공유되었는지 여부를 파악하고 기술지원이 가능한 응용 애플리케이션 보안 취약점이었다면, 기술지원이 가능한 부분을 검토하고 응용 애플리케이션의 보안 처리 조치가 있도록 지원해준다.

2. SW보안 결함 재발 방지 대책에 대한 평가를 수행한다.

보안 결함 재발 방지 대책에서 발견된 부적합 사항에 대해, 항구적인 재발 방지를 목적으로 방지 대책 심사를 수행하여야 하며 효과적인 수행 및 시정 조치 평가 방법을 수행한다.

가. 보안 결함의 현행 상태 파악

보안 취약점과 발생 일시를 명확히 파악하고 대상 시스템을 파악한다. 신규로 도입된 서버이고 보안 취약점이 완전히 조치가 된 경우, 주기적인 보안 취약점 점검의 원활한 수행을 위해 일정표에 다음 점검 일정을 갱신하고 추가한다.

나. SW보안 결함으로 침해가 예상되는 보안 사고의 내용을 파악한다.

결함으로 인한 침해사고 발생 원인과 시간 내용을 파악한다.

1) 침해 보안 사고의 발생 원인을 파악한다.

2) 침해 보안 사고의 조치 시간을 파악한다.

3) 침해 보안 사고의 조치 내용을 파악한다.

다. 보안 결함 재발 방지를 위한 내용을 파악한다.

3. SW보안 결함의 상태를 점검한다.

완전히 조치가 되었거나, 조직의 보안 정책에서 허용할 수준이 되었다면, 경영진 또는 최고 보안 담당자에게 보고하여 승인받고 이해관계자에게 서버의 보안 현황에 대한 통보를 한다.

가. 적합과 부적합 상태를 점검한다.

나. 시정 조치 계획을 재수립한다.

다. 시정 조치 계획을 제출하여 보고하고 검토받는다.

4. 부적합 사항의 시정 조치 절차를 점검한다.

가. 부적합 상태의 시정 조치 결과를 수신받는다.

나. 시정 조치 여부를 확인한다.

다. 시정 조치 결과를 통보하고 조치 계획을 수신하고 검토받는다.

5. 수정된 (임시 조치) 및 시정 조치 계획을 확인하고 승인받는다.

가. 수정(임시 조치)을 실시한다.

나. 부적합의 검토 및 발생하게 된 근본 원인을 파악한다.

1) 유사 프로세스를 점검하고 원인을 조사한다.

2) 유사 시스템을 포함한 원인을 조사한다.

다. 근본 원인을 제거한다.

라. 시정 조치 계획서 초안을 작성한다.

1) 초안을 작성하여 관련 이해관계자의 검토를 수행한다.

2) 검토 초안을 기반으로 관련 조직에 시정 조치를 요구한다.

3) 추진 일정 및 책임자를 명확히 명기한다.

4) 장기, 중기, 단기 순으로 시정 조치 계획을 수립한다.

6. 보안 결함 재발 방지의 시정 조치를 실시하고 유효성을 확인한다.
보안 결함의 심각도에 따라 담당자는 수립된 재발 방지 대책의 유효성을 확인하고 해당 부서의 담당자에게 통보하고 각 부서의 재발 방지 대책을 적용하도록 해야 한다.

가. 보안 결함 재발 방지를 위한 교육을 수행한다.
정보 보안 담당자는 재발 방지를 위하여 해당 부서의 담당자와 사용자를 대상으로 보안 사고 발생에 대한 대응 방안을 교육해야 하고 교육의 이수와 내용에 대한 기록을 유지관리 한다.

나. 보안 결함 재발 방지를 위한 재평가를 수행하고 결함 수정 여부를 재평가한다.
보안 결함 유형, 특성과 비중에 따라 기술 방법이 달라지는 경우가 있으나 의사결정자 입장에서 이해하기 쉽도록 작성을 한다.

보안 결함 재발 방지 계획

□ 사업장 개요

사업장명	웹 서비스 서버	보안 결함 일시	2026.04.14	보안 담당자	조채언
보안 결함 발생 개요	외부 공격자가 2026년 4월 14일, 웹 서버의 로그인 페이지를 대상으로 SQL 인젝션 공격을 수행했습니다. 공격자는 보안이 취약한 입력값 검증 과정을 이용해 악의적인 쿼리를 삽입했으며, 이를 통해 관리자 인증을 우회하거나 내부 DB 정보를 탈취하려 시도했습니다.				

□ 보안 결함 발생 원인 분석 및 재발 방지 대책

보안 결함 발생 원인	1. 물리적 요인: 서버 및 네트워크 장비의 보안 설정이 충분히 강화되지 않은 상태에서 운영되었고, 방화벽 및 웹 방화벽(WAF) 정책이 기본값으로 설정되어 외부 공격에 대한 탐지 및 차단 기능이 미흡하였다. 시스템 접근 통제 및 로그 관리 체계가 제대로 구축되지 않아 이상 행위에 대한 사전 탐지가 어려웠다. 2. 인적 요인: 개발자가 입력값 검증 및 필터링 로직을 충분히 구현하지 않아 악의적인 SQL 구문이 그대로 실행되었다. 또한, 보안 코딩 가이드라인에 대한 이해 부족으로 취약한 코드가 포함된 상태로 서비스가 배포되었고, 개발 및 운영 인력의 보안 인식 부족으로 코드 리뷰 및 취약점 점검 과정이 형식적으로 수행되었다. 3. 관리적 요인: 정기적인 보안 점검 및 취약점 분석 절차가 미흡하거나 수행되지 않았으며, 보안 정책 및 개발 표준이 명확하게 수립되지 않아 일관된 보안 관리가 이루어지지 않았다. 취약점 발견 후 대응 및 재발 방지 프로세스가 체계적으로 운영되지 않았고, 보안 교육 및 훈련 부족으로 조직 전반의 보안 대응 능력이 낮다.
재발 방지 대책	가. 단기 대책 - 모든 사용자 입력값에 대해 입력값 검증 및 필터링 로직을 즉시 적용하여 SQL Injection 공격을 차단한다. - 기존 SQL 쿼리를 Prepared Statement 방식으로 변경하여 동적 쿼리 실행을 방지한다. - 웹 방화벽(WAF) 및 침입 탐지 시스템(IDS/IPS)의 정책을 강화하여 의심 트래픽을 실시간으로 탐지 및 차단한다. - 취약점이 발견된 시스템에 대해 긴급 보안 패치 및 설정 변경을 수행한다. - 로그 모니터링을 강화하여 비정상 접근 시도를 즉시 확인하고 대응할 수 있도록 한다. 나. 장기 대책 - 개발 단계에서부터 보안을 고려한 Secure Coding 가이드라인을 수립하고 전사적으로 적용한다. - 정기적인 보안 취약점 점검 및 모의 해킹을 수행하여 잠재적인 취약점을 사전에 제거한다. - 코드 리뷰 및 배포 전 검증 절차를 강화하여 보안 취약 코드가 운영 환경에 반영되지 않도록 관리한다. - 개발자 및 운영자를 대상으로 정기적인 보안 교육 및 훈련을 실시하여 보안 인식을 향상시킨다. - 보안 사고 대응 프로세스를 체계화하고, 사고 발생 시 신속한 대응 및 재발 방지 체계를 구축한다. - 로그 관리 및 모니터링 시스템을 고도화하여 이상 행위를 자동으로 탐지하고 대응할 수 있도록 한다.

X. 정보보호 시스템 로그 점검 관리대장 작성

정보보호 시스템 관리자 계정 관리 대장

관리 담당자	정보보호 관리자
김민수	박지현

점검일	2026년 04월 14일

시스템명	로깅 관리	접근 로그	실패 로그	관리자 로그	보안 로그
웹 서버	양호	정상	일부 실패 시도 탐지	정상	SQL 인젝션 시도 탐지
DB 서버	양호	정상	X	정상	이상 X
WAS 서버	양호	정상	로그인 3회 실패 발생	정상	비정상 접근 X
FTP 서버	보통	정상	다수 실패 시도	관리자 접속 기록 있음	무차별 대입 공격 의심
DNS 서버	양호	정상	X	정상	이상 X
메일 서버	양호	정상	스팸 로그인 시도	정상	스팸 공격 탐지
백업 서버	양호	제한적 접근	X	정상	이상 X
내부 시스템	양호	정상	X	정상	내부 접근만 허용
테스트 서버	미흡	일부 누락	확인 필요	정상	로그 관리 필요
개발 서버	보통	정상	일부 실패 발생	정상	보안 로그 미흡

XI. 로그관리 점검 결과

□ 로그 관리 점검 결과 (이상/문제 발생 시 작성)

문제 발생 장비	FTP 서버 (File Transfer Server)
문제 내역	- 다수의 로그인 실패 로그가 반복적으로 발생함 - 특정 IP에서 지속적인 접속 시도가 확인됨 - 무차별 대입 공격(Brute Force Attack) 의심 정황 발견 - 로그 분석 결과 비인가 접근 시도가 다수 탐지됨
조치 내용/결과	- 해당 IP 주소를 방화벽에서 즉시 차단 조치 - FTP 서비스의 비밀번호 정책 강화 (복잡도 및 길이 증가) - 계정 잠금 정책 적용 (로그인 실패 횟수 제한 설정) - 관리자 계정 접근을 특정 IP로 제한 - 추가적인 비정상 접근 시도 여부 모니터링 강화하여 현재까지 추가적인 공격 시도 없음, 정상 운영 확인
향후 대책	- 침입 탐지 시스템(IDS/IPS) 및 웹 방화벽(WAF) 정책 강화 - 정기적인 로그 점검 및 이상 징후 모니터링 체계 구축 - FTP 서비스 대신 보안성이 강화된 SFTP 사용 검토 - 사용자 계정 및 접근 권한 관리 강화 - 관리자 및 사용자 대상 보안 교육 실시
IT보안 관리자 검토 의견	- 본 사고는 외부 공격에 의한 무차별 대입 공격으로 판단됨 - 초기 대응이 신속하게 이루어져 피해는 발생하지 않음 - 향후 유사 공격에 대비한 보안 정책 강화 필요
관리 담당자 검토 의견	- 계정 보안 설정 미흡으로 공격 시도가 발생한 것으로 판단됨 - 정기적인 보안 점검 및 계정 관리 강화 필요 - 재발 방지를 위해 지속적인 모니터링 및 점검 수행 예정